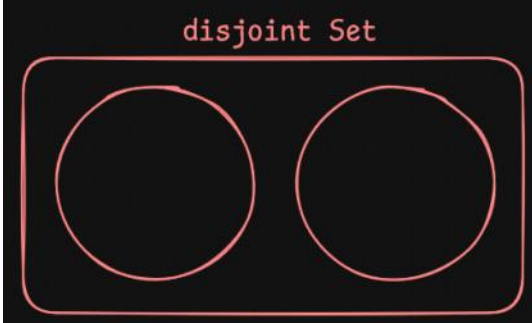
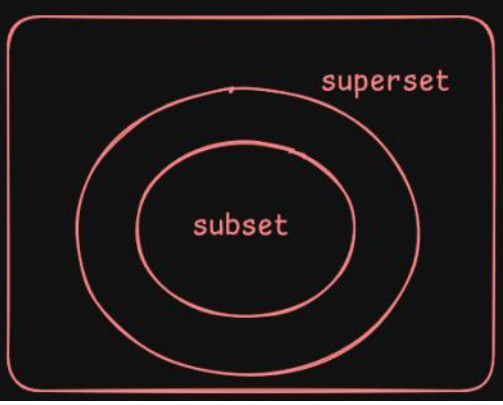
Data Structures-II

Session Objectives

- Understand what sets are.
- Understand common methods and operations associated with sets.
- Understand the comparison between lists, tuples and sets.
- Understand what dictionaries are.
- Understand common methods and operations associated with dictionaries.
- Understand the comparison between lists, tuples, sets and dictionaries.

Joining Sets





```python
# Union() -> (| or .union())
setA = {1,2,3,4,5,6,7,8,8,7,6,5,4,4,3} # {1,2,3,4,5,6,7,8}
setB = {3,4,5,6,7,8,9,3,4,} # {3,4,5,6,7,8,9}
union_set = setA | setB # A U B
print(union_set) # {1,2,3,4,5,6,7,8,9}
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```python
result = setA.union(setB)
print(result) #  {1,2,3,4,5,6,7,8,9}
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```python
# Advance Union Concepts
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
print(setA | setB | setC)
print(setA.union(setB).union(setC))
```

```
{'s', 'q', 'p', 'x', 'r', 'a', 'e', 'c', 'b', 'y', 'z', 'd'}
{'s', 'q', 'p', 'x', 'r', 'a', 'e', 'c', 'b', 'y', 'z', 'd'}
```

```python
# Intersection (& or .intersection()) -> Common Elements
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
print(setA & setB) # {'a','b','c'}
```

```
{'c', 'a', 'b'}
```

```python
print(setA.intersection(setB).intersection(setC)) # {'a','c'}
```

```
{'c', 'a'}
```

```python
print(setA & setB & setC) # {'a','c'}
```

```
{'c', 'a'}
```

```python
# difference (- or .difference) # Element in A But not in B
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
print(setA - setB) # {'d','e'}
print(setA.difference(setB)) # {'d','e'}
```

```
{'e', 'd'}
{'e', 'd'}
```

```python
print(setB - setA) # pqrs
print(setB.difference(setA)) # pqrs
```

```
{'q', 'p', 'r', 's'}
{'q', 'p', 'r', 's'}
```

```python
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
print(setB - setA - setC) # pq
print(setB.difference(setA).difference(setC)) #  pq
```

```
{'q', 'p'}
{'q', 'p'}
```

```python
# Symmetric Difference (^ or .symmetric_difference())
# Elements in either SetA or SetB, but not in both
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
print(setA ^ setB) # depqrs
print(setA.symmetric_difference(setB)) # depqrs
```

```
{'s', 'q', 'p', 'r', 'e', 'd'}
{'s', 'q', 'p', 'r', 'e', 'd'}
```

```python
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
print(setA ^ setB ^ setC) # depqrs ^ {'x','y','r','s','a','c','z'}
# depqxyzac
print(setA.symmetric_difference(setB).symmetric_difference(setC))
```

```
{'p', 'a', 'e', 'y', 'z', 'd', 'q', 'x', 'c'}
{'p', 'a', 'e', 'y', 'z', 'd', 'q', 'x', 'c'}
```

```python
# .difference_update()
# shortcut : a -= b|c -> setA = setA - (setB | setC)
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
# (setB | setC) # {abcpqrsxyz}
# setA - (setB | setC) => {abcde} - {abcpqrsxyz} # {de}
setA = setA - (setB | setC)
print(setA) # {de}
```

```
{'e', 'd'}
```

```python
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA.difference_update(setB , setC)
print(setA) # {de}
```

```
{'e', 'd'}
```

```python
# .intersection_update()
# shortcut : a &= b -> a = a&b
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA.intersection_update(setB) # {abc}
print(setA) # {abc}
```

```
{'c', 'a', 'b'}
```

```python
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA = setA & setB
print(setA)
```

```
{'c', 'a', 'b'}
```

```python
# .intersection_update()
# shortcut : a &= b&c -> a = a&(b&c)

setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
# setB & setC {rsac}
setA.intersection_update(setB,setC) # setA & {rsac} -> {ac}
print(setA)
```

```
{'c', 'a'}
```

```python
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA = setA & (setB & setC)
print(setA)
```

```
{'c', 'a'}
```

```python
# isdisjoint() -> Boolean Return [T/F]
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setD = {'k','m'}
print(setA.isdisjoint(setB)) # False
print(setA.isdisjoint(setD)) # True
```

```
False
True
```

```python
# issubset() or issuperset() -> Boolean Return [T/F]
set1 = {1,2,3,4,5}
set2 = {1,2,3,4,5,6,7,8,9}
print(set1.issubset(set2)) # True as set1 is a subset of set2
print(set2.issuperset(set1)) # True as set2 is a superset of set1
```

```
True
True
```

```python
print(set2.issubset(set1)) # False as set1 is a subset of set2
print(set1.issuperset(set2)) # False as set2 is a superset of set1
```

```
False
False
```

```python
# issubset() or issuperset() -> Boolean Return [T/F]
set1 = {1,2,3,4,5,'a','b','c'}
set2 = {1,2,3,4,5,6,7,8,9}
print(set1.issubset(set2)) # False as set1 has few more elements ['a','b','c']
print(set2.issuperset(set1)) # False as set1 has few more elements ['a','b','c']
```

```
False
False
```

```python
# symmetric_difference_update()
# shortcut : a ^= b
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA.symmetric_difference_update(setB) # Either in A or B , but not in both
print(setA) # depqrs
```

```
{'s', 'q', 'p', 'r', 'e', 'd'}
```

```python
# symmetric_difference_update()
# shortcut : a ^= b
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA = setA ^ setB # Either in A or B , but not in both
print(setA) # depqrs
```

```
{'s', 'q', 'p', 'r', 'e', 'd'}
```

```python
# symmetric_difference_update()
# shortcut : a ^= (b ^ c)
# a = a ^ (b ^ c)
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA = setA ^ (setB ^ setC)
# (setB ^ setC) -> {pqbxyz}
# setA ^ {pqbxyz} -> {abcde} ^ {pqbxyz} -> {acdepqxyz}
print(setA) # {acdepqxyz}
```

```
{'p', 'a', 'e', 'y', 'z', 'd', 'x', 'q', 'c'}
```

```python
# .update() |
# shortcut : a |= b|c
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA.update(setB,setC)
print(setA)
```

```
{'s', 'q', 'p', 'x', 'r', 'a', 'e', 'c', 'b', 'y', 'z', 'd'}
```

```python
setA = {'a','b','c','d','e'}
setB = {'p','q','r','s','a','b','c'}
setC = {'x','y','r','s','a','c','z'}
setA = setA | setB | setC
print(setA)
```

```
{'s', 'q', 'p', 'x', 'r', 'a', 'e', 'c', 'b', 'y', 'z', 'd'}
```

```python
# Copying the set -> Shallow Copy -> .copy() , set()
day_set = {'Mon','Tue','Wed','Thurs','Fri','Sat','Sun'}
copy_day_set = day_set.copy() # shallow Copy
print(id(day_set))
print(id(copy_day_set))
```

```
1681904212832
1681904215520
```

```python
copy_day_set.add("Jan")
print(day_set)
print(copy_day_set)
```

```
{'Thurs', 'Sun', 'Fri', 'Wed', 'Mon', 'Tue', 'Sat'}
{'Thurs', 'Sun', 'Jan', 'Fri', 'Wed', 'Mon', 'Tue', 'Sat'}
```

```python
# set() constructor -> typecast -> shallow copy
day_set = ('Mon','Tue','Wed','Thurs','Fri','Sat','Sun')
another_day_set = set(day_set) # shallow Copy
print(id(day_set))
print(id(another_day_set))
```

```
1681925017888
1681904212832
```

```python
another_day_set.discard('Sat')
another_day_set.discard('Sun')
print(day_set)
print(another_day_set)
```

```
('Mon', 'Tue', 'Wed', 'Thurs', 'Fri', 'Sat', 'Sun')
{'Thurs', 'Wed', 'Mon', 'Fri', 'Tue'}
```

What is Dictionary in Python?

A Dictionary is :

1. `Ordered` : Items have guaranteed sequence.
2. `Mutable` : You can change, add or remove items.
3. `Collection of Key-Value Pair` : Each Key is Unique and maps to its value.

```python
_dict = {}
print(_dict)
print(type(_dict))
```

```
{}
<class 'dict'>
```

```python
# Dictionary -> When defining duplicate keys , only the last occurrence is kept.
student_details = {
    'name' : 'Siddharth Sharan',
    'age' : 29,
    'gender' : 'Male',
    'city' : 'Jaipur',
    'state' : 'Rajasthan',
    'country' : 'India',
    'course' : 'Data Science',
    'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
    'course' : 'Data Science with GenAI'
}
student_details
```

```
{'name': 'Siddharth Sharan',
 'age': 29,
 'gender': 'Male',
 'city': 'Jaipur',
 'state': 'Rajasthan',
 'country': 'India',
 'course': 'Data Science with GenAI',
 'skills': ['Python', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP']}
```

```
# You can also use tuple as key
_dict = {
    ('stud1','stud2') : 98,
    ('stud3','stud4') : 77,
    ('stud5','stud6') : 81,
    ('stud7','stud8') : 91,
    ('stud9','stud10') : 89
}
_dict
```

```
{('stud1', 'stud2'): 98,
 ('stud3', 'stud4'): 77,
 ('stud5', 'stud6'): 81,
 ('stud7', 'stud8'): 91,
 ('stud9', 'stud10'): 89}
```

```
# dict() constructor
_dict = dict(name = "Pratyush" , age = 27 , gender = "Male" , state = "Delhi" , country = "India")
_dict
```

```
{'name': 'Pratyush',
 'age': 27,
 'gender': 'Male',
 'state': 'Delhi',
 'country': 'India'}
```

```
# With a List of List -> Nested List
list_of_list = [
    ['name' , 'Paras'],
    ['age' , 25],
    ['gender' , 'Male'],
    ['city' , 'Indore'],
    ['state' , 'Madhya Pradesh'],
    ['country' , 'India']
]
_dict = dict(list_of_list)
_dict
```

```
{'name': 'Paras',
 'age': 25,
 'gender': 'Male',
 'city': 'Indore',
 'state': 'Madhya Pradesh',
 'country': 'India'}
```

```python
# With a List of Tuple -> Nested List
list_of_tuple = [
    ('name' , 'Paras'),
    ('age' , 25),
    ('gender' , 'Male'),
    ('city' , 'Indore'),
    ('state' , 'Madhya Pradesh'),
    ('country' , 'India')
]
_dict = dict(list_of_tuple)
_dict
```

```
{'name': 'Paras',
 'age': 25,
 'gender': 'Male',
 'city': 'Indore',
 'state': 'Madhya Pradesh',
 'country': 'India'}
```

```python
# With a Tuple of Tuple -> Nested Tuple
tuple_of_tuple = (
    ('name' , 'Paras'),
    ('age' , 25),
    ('gender' , 'Male'),
    ('city' , 'Indore'),
    ('state' , 'Madhya Pradesh'),
    ('country' , 'India')
)
_dict = dict(tuple_of_tuple)
_dict
```

```
{'name': 'Paras',
 'age': 25,
 'gender': 'Male',
 'city': 'Indore',
 'state': 'Madhya Pradesh',
 'country': 'India'}
```

```python
_list = [[11,'rupees']]
_dict = dict(_list)
_dict
```

```
{11: 'rupees'}
```

```python
# Dictionary -> When defining duplicate keys , only the last occurrence is kept.
student_details = {
    'name' : 'Siddharth Sharan',
    'age' : 29,
    'gender' : 'Male',
    'city' : 'Jaipur',
    'state' : 'Rajasthan',
    'country' : 'India',
    'course' : 'Data Science',
    'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
    'course' : 'Data Science with GenAI'
}
student_details
```

```
{'name': 'Siddharth Sharan',
 'age': 29,
 'gender': 'Male',
 'city': 'Jaipur',
 'state': 'Rajasthan',
 'country': 'India',
 'course': 'Data Science with GenAI',
 'skills': ['Python', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP']}
```

```python
# Nested Dictionaries
student_record = {
    "Student1" : {
        'name' : 'Siddharth Sharan',
        'age' : 29,
        'gender' : 'Male',
        'city' : 'Jaipur',
        'state' : 'Rajasthan',
        'country' : 'India',
        'course' : 'Data Science',
        'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
        'course' : 'Data Science with GenAI'
    },
    "Student2" : {
        'name' : "Ankit Agrawal",
        'age' : 32,
        'gender' : 'Male',
        'city' : 'Gurugram',
        'state' : 'Haryana',
        'country' : 'India',
        'course' : 'Data Science',
        'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
    }
}
student_record
```

```
{'Student1': {'name': 'Siddharth Sharan',
  'age': 29,
  'gender': 'Male',
  'city': 'Jaipur',
  'state': 'Rajasthan',
  'country': 'India',
  'course': 'Data Science with GenAI',
  'skills': ['Python', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP']},
 'Student2': {'name': 'Ankit Agrawal',
  'age': 32,
  'gender': 'Male',
  'city': 'Gurugram',
  'state': 'Haryana',
  'country': 'India',
  'course': 'Data Science',
  'skills': ['Python', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP']}}
```

```python
# Understanding common methods and operations associated with dictionaries.
# len()
print(len(student_record)) # 2
```

```
2
```

```python
print(len(student_details)) # 8
```

```
8
```

```python
print(len(student_record['Student1'])) # 8
```

```
8
```

```python
# Accessing an Element
# Use Square Bracket Dict['Key']
student_details = {
    'name' : 'Siddharth Sharan',
    'age' : 29,
    'gender' : 'Male',
    'city' : 'Jaipur',
    'state' : 'Rajasthan',
    'country' : 'India',
    'course' : 'Data Science',
    'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
    'course' : 'Data Science with GenAI'
}
student_details['skills']
```

```
['Python', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP']
```

```python
type(student_details['skills']) # 'List'
```

```
list
```

```python
student_details['course'] # Last updated value
# 'Data Science with GenAI'
```

```
'Data Science with GenAI'
```

```python
# Nested Dictionaries
student_record = {
    "Student1" : {
        'name' : 'Siddharth Sharan',
        'age' : 29,
        'gender' : 'Male',
        'city' : 'Jaipur',
        'state' : 'Rajasthan',
        'country' : 'India',
        'course' : 'Data Science',
        'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
        'course' : 'Data Science with GenAI'
    },
    "Student2" : {
        'name' : "Ankit Agrawal",
        'age' : 32,
        'gender' : 'Male',
        'city' : 'Gurugram',
        'state' : 'Haryana',
        'country' : 'India',
        'course' : 'Data Science',
        'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
    }
}
student_record['Student2']['name'] # 'Ankit Agrawal'
```

```
'Ankit Agrawal'
```

```python
student_record['Student1']['state'] # 'Rajasthan'
```

```
'Rajasthan'
```

```python
student_record['Student2']['skills'][-3:] # ['ML','DL','NLP']
```

```
['ML', 'DL', 'NLP']
```

```python
_dict = {
    ('stud1','stud2') : 98,
    ('stud3','stud4') : 77,
    ('stud5','stud6') : 81,
    ('stud7','stud8') : 91,
    ('stud9','stud10') : 89
}
_dict[('stud9','stud10')] # 89
```

```
89
```

```python
_dict[('stud11','stud12')]
# KeyError: ('stud11', 'stud12')
```

```python
# .get(key , default) -> It will safely retrieve the value; returns None or a default if the key not found.
# Accessing an Element
# Use Square Bracket Dict['Key']
student_details = {
    'name' : 'Siddharth Sharan',
    'age' : 29,
    'gender' : 'Male',
    'city' : 'Jaipur',
    'state' : 'Rajasthan',
    'country' : 'India',
    'course' : 'Data Science',
    'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
    'course' : 'Data Science with GenAI'
}
student_details.get('course','AI Engineer')
```

```
'Data Science with GenAI'
```

```python
student_details.get('email' , 'xyz@gmail.com')
```

```
'xyz@gmail.com'
```

```python
student_details.get('skills')
```

```
['Python', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP']
```

```python
student_details.get('phone_number') # None
```

```python
print(student_details.get('phone_number'))
```

```
None
```

```python
# Nested Dictionaries
student_record = {
    "Student1" : {
        'name' : 'Siddharth Sharan',
        'age' : 29,
        'gender' : 'Male',
        'city' : 'Jaipur',
        'state' : 'Rajasthan',
        'country' : 'India',
        'course' : 'Data Science',
        'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
        'course' : 'Data Science with GenAI'
    },
    "Student2" : {
        'name' : "Ankit Agrawal",
        'age' : 32,
        'gender' : 'Male',
        'city' : 'Gurugram',
        'state' : 'Haryana',
        'country' : 'India',
        'course' : 'Data Science',
        'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
    }
}
student_record.get('Student2',{}).get('name','Unknown') # 'Ankit Agrawal'
```

```
'Ankit Agrawal'
```

```python
student_record.get('Student7',{}).get('name','Unknown') # 'Unknown'
```

```
'Unknown'
```

```python
# .keys() -> Returns all the keys from the given dictionary
# .values() -> Returns all the values from the given dictionary
# .items() -> Returns all the (key,value) from the given dictionary
student_record.keys()
```

```
dict_keys(['Student1', 'Student2'])
```

```python
student_record['Student1'].keys()
```

```
dict_keys(['name', 'age', 'gender', 'city', 'state', 'country', 'course', 'skills'])
```

```python
student_record['Student2'].keys()
```

```
dict_keys(['name', 'age', 'gender', 'city', 'state', 'country', 'course', 'skills'])
```

```python
student_details.keys()
```

```
dict_keys(['name', 'age', 'gender', 'city', 'state', 'country', 'course', 'skills'])
```

```python
student_details.values()
```

```
dict_values(['Siddharth Sharan', 29, 'Male', 'Jaipur', 'Rajasthan', 'India', 'Data Science with GenAI', ['Pyth
on', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP']])
```

```
student_details.items() # key-value pair stored as a tuple in a list
```

```
dict_items([('name', 'Siddharth Sharan'), ('age', 29), ('gender', 'Male'), ('city', 'Jaipur'), ('state', 'Raja
sthan'), ('country', 'India'), ('course', 'Data Science with GenAI'), ('skills', ['Python', 'Numpy', 'Pandas',
'Statistics', 'ML', 'DL', 'NLP'])])
```

```
# .setdefault() -> Retrieves the value if the key exist , else insert the key with default
# if no default provided , insert None.
student_details = {
    'name' : 'Siddharth Sharan',
    'age' : 29,
    'gender' : 'Male',
    'city' : 'Jaipur',
    'state' : 'Rajasthan',
    'country' : 'India',
    'course' : 'Data Science',
    'skills' : ['Python','Numpy','Pandas','Statistics','ML','DL','NLP'],
    'course' : 'Data Science with GenAI'
}
student_details.setdefault('course') # 'Data Science with GenAI'
```

```
'Data Science with GenAI'
```

```
student_details.setdefault('course' , 'AI Engineer') # 'Data Science with GenAI'
```

```
'Data Science with GenAI'
```

```
student_details.setdefault('email' , 'xyz@gmail.com') # 'xyz@gmail.com'
```

```
'xyz@gmail.com'
```

```
student_details
```

```
{'name': 'Siddharth Sharan',
 'age': 29,
 'gender': 'Male',
 'city': 'Jaipur',
 'state': 'Rajasthan',
 'country': 'India',
 'course': 'Data Science with GenAI',
 'skills': ['Python', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP'],
 'email': 'xyz@gmail.com'}
```

```
print(student_details.setdefault('phone_number' )) # None
```

```
None
```

```
student_details
```

```
{'name': 'Siddharth Sharan',
 'age': 29,
 'gender': 'Male',
 'city': 'Jaipur',
 'state': 'Rajasthan',
 'country': 'India',
 'course': 'Data Science with GenAI',
 'skills': ['Python', 'Numpy', 'Pandas', 'Statistics', 'ML', 'DL', 'NLP'],
 'email': 'xyz@gmail.com',
 'phone_number': None}
```

```
student_details.setdefault('email' , 'xyz@gmail.com') # 'xyz@gmail.com'
```

```
'xyz@gmail.com'
```

```
student_details
```

```
{'name': 'Siddh
 'age': 29,
 'gender': 'Male',
```

```
Signature: student_details.setdefault(key, default=None, /)
Docstring:
Insert key with a value of default if key is not in the dictionary.

Return the value for key if key is in the dictionary, else default.
Type:      builtin_function_or_method
```